

SQLAlchemy Validation

Catching the data corruption that migration tools report as success

Hubert Dec · Patryk Dzik · Adrian Czapka · Dominik Dziedzic · Beniamin Sujka

The migration succeeded. The data did not.

Six documented incidents where no error fired — yet the data came out wrong.

The dangerous migration failure is the one that **does not** announce itself. No exception, no alert, the row count matches, and the tooling reports success. The corruption surfaces days or weeks later — through a customer, an analyst, or a regulator.

Organisation	What went wrong	Severity	Found by
Netflix Billing	Oracle NUMBER had no exact MySQL type — silent truncation risk	MEDIUM	Pre-migration audit
Insurance (EU)	127 000 policies from the 1940s migrated as issued in 2040	MEDIUM	User, 3 weeks later
MySQL → Postgres	87 % of timestamps shifted by a whole timezone offset	HIGH	Customer report
OpenAI / Redis	Cache resequencing returned data to the wrong user sessions	HIGH	User reports
TSB Bank	1.3 billion records corrupted; customers saw strangers' accounts	CRITICAL	Customers, 20 min later
TUI Aviation	Title “Miss” remapped to child mass — load sheet 1 244 kg light	CRITICAL	Post-flight audit

Every one passed its structural checks. The only failing check was the one nobody ran: **does this value still mean what it meant before?**

The gap in standard migration tooling

Schema constraints and ORM validators guard new writes — not the rows already loaded.

1 • Validation confirmed movement, not meaning

Row counts matched and logs were empty — yet none of that verifies a value still carries the **same meaning** it had in the source.

3 • Bulk loads bypass the ORM

`pandas.to_sql`, `COPY`, and raw SQL write straight to the table — skipping every `@validates` hook and often disabling constraints for speed.

2 • The errors were self-concealing

A corrupted timestamp is still a valid timestamp; a misdated policy is still a well-formed date. The corruption hides **inside** normal-looking data.

4 • Detection came from users, not monitoring

TSB's customers reported wrong accounts within 20 minutes of go-live. In no case did internal monitoring catch the corruption first.

Millions of migrated rows can hold violations the model would reject on a fresh insert — and **nothing in the normal toolchain re-checks them.**

How we solve it

One declarative engine, four proven tools — no parallel config to drift out of sync.

SQLAlchemy `src/engine.py`

Rules live on the ORM column as Annotated markers. The engine reads them and runs **set-based SQL** against the live table — streamed with `yield_per`, constant memory at any scale.

pandas `src/pandas_.py`

`errors_to_dataframe / summarize_errors` turn raw violations into a tidy report — grouped by kind and column for instant triage.

pytest `pytest plugin`

A registered plugin exposes `assert_table_valid(session, Model)` — one assertion turns the whole audit into a pass/fail CI gate.

Streamlit `dashboard/app.py`

A live dashboard with auto-refresh and a configurable connection string — paste any SQLAlchemy URL and watch violations in real time.

Declare the rule once, on the column

Same file, same code review — the validation lives next to the data it protects.

```
from typing import Annotated
from sqlalchemy.orm import Mapped, mapped_column
from src.metadata import (
    Required, Unique, FloatMin, FloatMax,
    UniqueComposite, NoNaN,
)

class User(Base):
    __tablename__ = "users"
    id: Mapped[int]
    email: Mapped[Annotated[str, Required(), Unique()]]
    age: Mapped[Annotated[int, Required(),
                          FloatMin(0), FloatMax(120)]]
    country: Mapped[Annotated[str, Required(),
                              UniqueComposite("country", "city")]]
    city: Mapped[Annotated[str, Required()]]
    score: Mapped[Annotated[float, NoNaN()]]
```

Markers available in v1

Marker	Meaning
Required()	Value must not be NULL
Nullable()	Explicitly permits NULL
Unique()	Values must be unique
UniqueComposite	Multi-column unique rule
FloatMin(n)	Numeric lower bound
FloatMax(n)	Numeric upper bound
NoNaN()	Reject IEEE NaN in a float

A non-optional `Mapped[T]` implies `Required()` automatically; `Mapped[T | None]` skips the null check.

Run it — three lines

SQL-first, streaming, two passes — no ORM overhead, no full table held in memory.

```
from sqlalchemy.orm import Session
from src.engine import MigrationValidator
from src.pandas_ import (
    errors_to_dataframe, summarize_errors,
)

with Session(engine) as session:
    errors = MigrationValidator(session) \
        .validate(User)

report = errors_to_dataframe(errors)
summary = summarize_errors(report)
```

Example summary output

kind	column	count
null	email	52
duplicate	email	50
float_min	age	20

Pass 1 • Field rules

Each marker emits one SELECT for **only** the violating rows (e.g. WHERE age < 0), streamed with yield_per=5000.

Pass 2 • Duplicate rules

Unique / UniqueComposite targets are gathered into one GROUP BY ...
HAVING COUNT(*) > 1 per constraint.

Python only where SQL can't

NoNaN() pre-filters non-null rows in SQL, then checks math.isnan() in Python — NaN comparison is not portable across engines.

How it compares to the alternatives

Other tools either ignore the rows already in the table, or drift away from the model.

Approach	Audits existing rows?	Declared on the model?	Needs data in memory?
SQLAlchemy #"@validates"	No (new writes only)	Yes	No
DB CheckConstraint	No (new writes only)	Yes	No
Pandera schema	Yes	No (separate)	Yes (full DataFrame)
Great Expectations	Yes	No (suite)	Yes
SQLAlchemy Validation	Yes (set-based SQL)	Yes	No (streamed)

@validates — runs only on ORM flush, never on a bulk load:

```
class User(Base):
    @validates("age")
    def check_age(self, key, value):
        if value < 0:
            raise ValueError("bad age")
        return value  # skipped by to_sql / COPY
```

Our validation — one marker, audits every row already in the table:

```
class User(Base):
    age: Mapped[Annotated[int, FloatMin(0)]]

# later, against the live DB:
MigrationValidator(session).validate(User)
```

Gate the pipeline, watch it live

The same models power a CI assertion and a real-time dashboard.

pytest — pass/fail gate

```
# test_migration.py
from src.pytest_validation_plugin import (
    assert_table_valid,
)

def test_users_after_migration(session):
    assert_table_valid(session, User)

def test_policies_after_migration(session):
    assert_table_valid(session, Policy)
```

On failure the assertion prints the total error count and the summary DataFrame.

Add it to CI when...

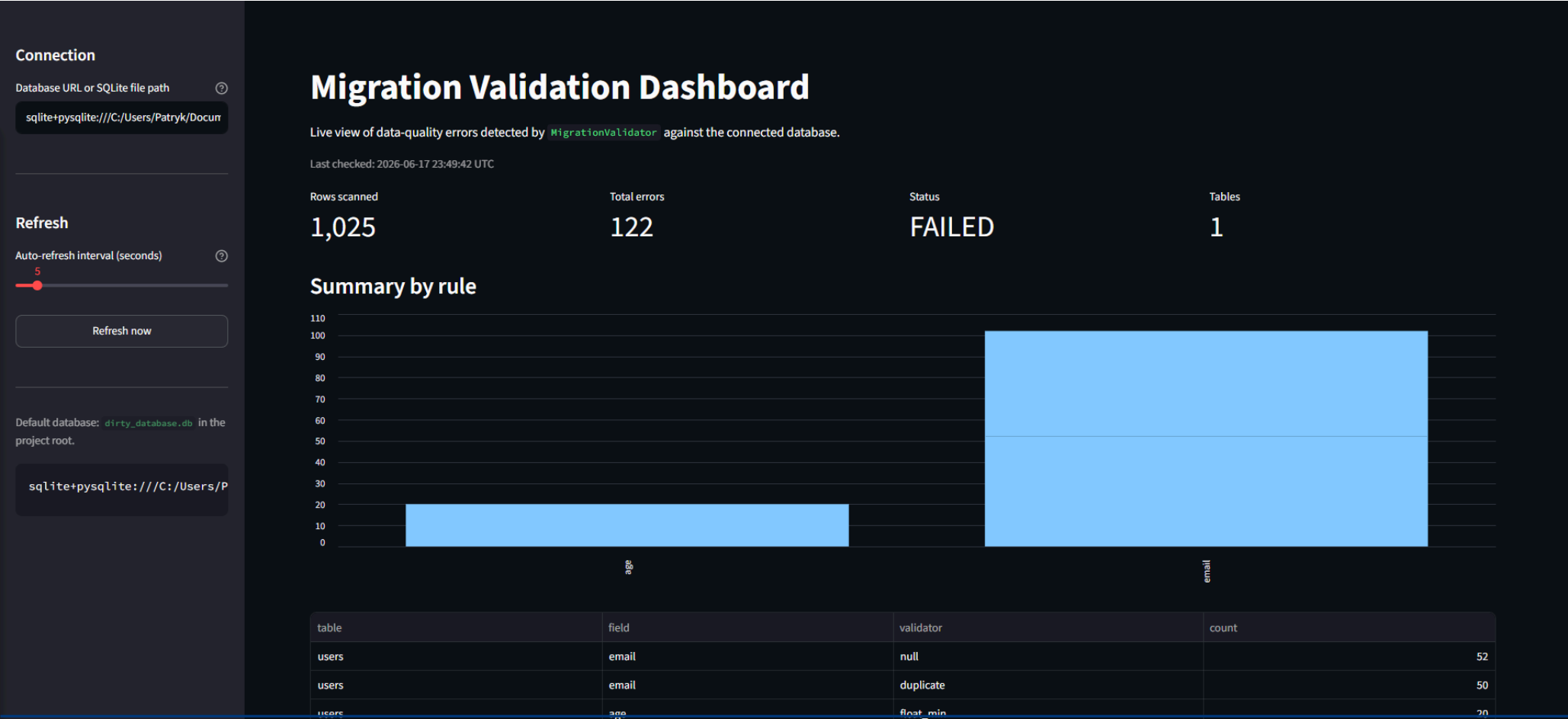
- an ETL / bulk load ran outside the ORM
- staging data mirrors production volume
- the schema changed since last migration
- a hotfix touched legacy rows
- cut-over needs a sign-off gate

Streamlit dashboard

`uv run streamlit run dashboard/app.py` — paste any SQLAlchemy URL, auto-refresh, and triage violations by kind and column without writing a query.

Streamlit dashboard — overview

Live KPIs, rule summary, and pass/fail status on any SQLAlchemy connection.



Drill into every error

Filter by table, field, and validator — then inspect row-level violations.

Connection

Database URL or SQLite file path ⓘ

sqlite+pysqlite:///C:/Users/Patryk/Docum

Refresh

Auto-refresh interval (seconds) ⓘ

5

Refresh now

Default database: dirty_database.db in the project root.

sqlite+pysqlite:///C:/Users/P

users	age	float_min	20
-------	-----	-----------	----

Error details

Validator

duplicate × float_min × null ×

Field

age × email ×

Table

users ×

table	id	field	error	validator
users	15	email	Field 'email' must not be NULL	null
users	53	email	Field 'email' must not be NULL	null
users	58	email	Field 'email' must not be NULL	null
users	60	email	Field 'email' must not be NULL	null
users	64	email	Field 'email' must not be NULL	null
users	66	email	Field 'email' must not be NULL	null
users	73	email	Field 'email' must not be NULL	null
users	103	email	Field 'email' must not be NULL	null
users	110	email	Field 'email' must not be NULL	null
users	117	email	Field 'email' must not be NULL	null

Showing 122 of 122 errors

Cheap to run, expensive to skip

Benchmarks from examples/_bench_timing.py — SQLite, four rules, 10 250 rows.

≈ 0.03 s

Automated check
vs 80 min by hand

160 000×

Faster than the
manual equivalent

≈ 21 h

Engineer time saved
per year (base case)

£600

One-time setup
(8 h to onboard)

Why it matters

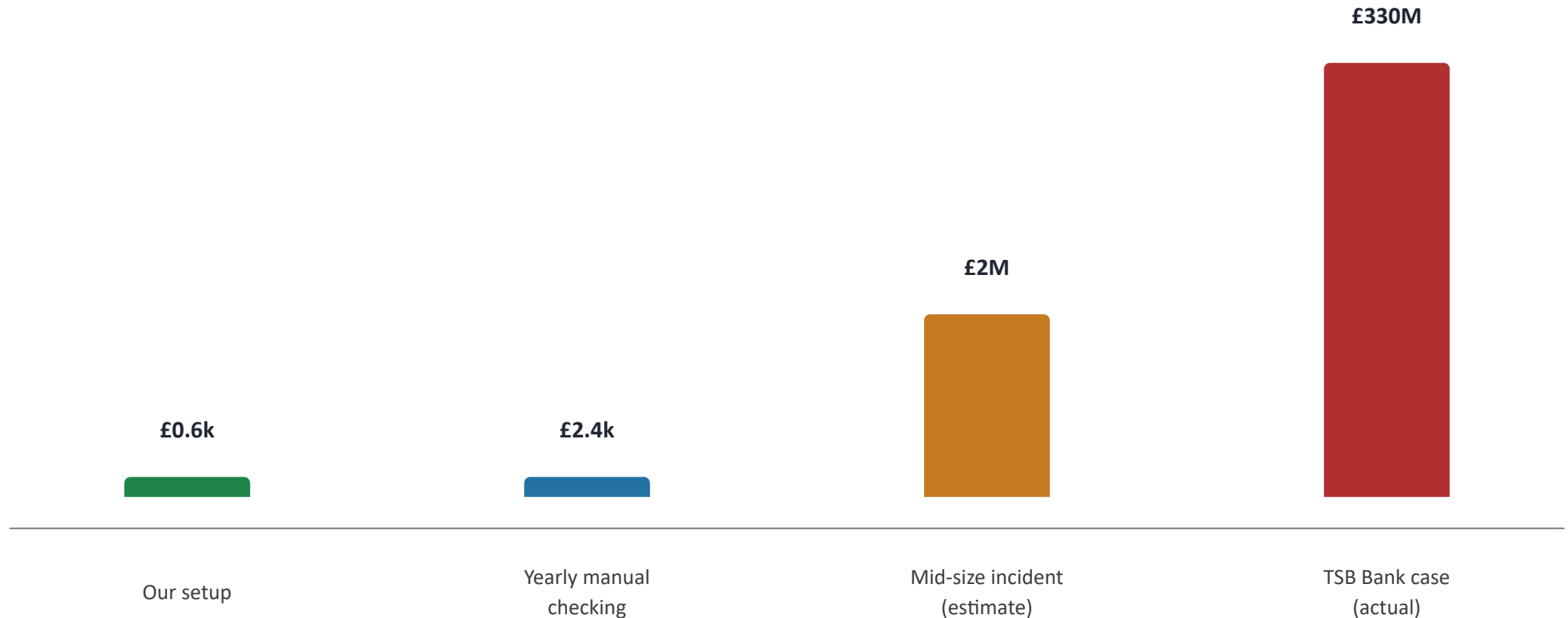
Manual checking grows with the data; the automated check stays under a second at every size tested. The real value, though, is **avoiding the late surprise** — not the time saved.

Defects caught in the demo

On 1 025 deliberately dirty rows: **52 missing, 46 duplicate, 20 out-of-range** — 118 bad records that a row-count check would have waved straight through.

The cost of a missed defect

Setup is a rounding error next to a single real incident.



A red test costs an engineer an hour. The same defect post-go-live cost TSB **£330 million**, 80 000 customers, and an £48.65 M regulatory fine.

Summary

Structural integrity today; semantic checks tomorrow — always declared on the model.

Why it exists

- “Successful” migrations still corrupt data
- Row counts never verify field-level correctness
- Bulk loads bypass ORM validation
- Users find the problem before monitoring does

How to adopt it

1. Annotate mapped columns with markers
2. Run `MigrationValidator(session).validate(Model)`
3. Review `errors_to_dataframe / summarize_errors`
4. Gate cut-over with `assert_table_valid`

What it covers now

- NULL violations — `Required()`
- Out-of-range — `FloatMin, FloatMax`
- Uniqueness — `Unique, UniqueComposite`
- IEEE NaN in floats — `NoNaN()`

Where it’s going

- Category / ENUM remapping validators
- Cross-table referential integrity
- Timezone-aware timestamp comparison
- Custom domain markers via `parser.py`

```
uv run pytest · uv run python examples/validate_dirty_database.py · uv run streamlit run dashboard/app.py
```